TECHNICAL NOTE 12

PRACTICAL IMPLEMENTATION OF THE
AGENTIC CI WORKFLOW

Claude Code Prompts + GitHub + Google Cloud Run

Live Data Retrieval, Validation, and Configurable Case Studies

Prof. Cesar Moreno Pascual PhD.

Corporate Intelligence Course | Universidad Francisco de Vitoria

2025-2026 Academic Year

Table of Contents

How to Use This Document 3
1. Module 0: Project Foundation, Dockerfile, and First Deployment 4
2. Module 1: OSINT Collector with Live Data Retrieval and Validation 7
3. Module 2: AMC Analyst Agent 9
4. Module 3: Scenario Planner, Exchange Value, and ECOMO 10
5. Module 4: Anti-Hallucination Layer 11
6. Module 5: LangGraph Orchestration Pipeline 12
7. Module 6: War Game Simulator 13
8. Module 7: CI Reporter and Executive Brief 14
9. Deliverables Checklist 15
10. Complete Settings Reference 16
References 18

How to Use This Document

This technical note builds a complete CI agent system module by module. You type prompts into Claude Code — it generates everything. After each module you push to GitHub and deploy to Google Cloud Run. The system retrieves **live data from real sources** — nothing is hardcoded. The default case is BBVA vs Revolut, but you can change it to any companies in any sector from the Settings panel.

Step	What You Do	What Claude Code Does	Result
1. Prompt	Type the blue box into Claude Code	Generates all code and files	Working module locally
2. Settings	Review the orange settings box	Creates editable settings panel	Modify prompts, companies, thresholds
3. Test	Run the green checkpoint	Validates the module	Verified component
4. Deploy	Type the purple deploy box	Pushes to GitHub + deploys to Cloud Run	Live on the internet

Four Colour-Coded Boxes

BLUE = Claude Code prompts. Type EXACTLY into Claude Code.
ORANGE = Configurable settings. Editable in the frontend without code.
GREEN = Checkpoints. Verify before continuing.
PURPLE = Deploy. Push to GitHub + deploy to Google Cloud Run.

Prerequisites: Complete the three setup guides first: (1) Claude Code Setup, (2) GitHub Setup, (3) Google Cloud Run Setup.

1. Module 0: Project Foundation, Dockerfile, and First Deployment

This module creates the project structure, the CI framework context (WITHOUT hardcoded data), the Dockerfile, a FastAPI backend, a React frontend with a configurable case selector, and deploys everything to Google Cloud Run. The system retrieves all data live from real sources.

CLAUDE CODE PROMPT 1

Project Structure, CLAUDE.md (Frameworks Only), Dockerfile, and Configurable Case Selector

Create a project called bbva-revolut-ci for a Competitive Intelligence agent system.

1. Create CLAUDE.md with CI FRAMEWORKS ONLY (no hardcoded company data):
   - AMC Framework (TN01): Score Awareness (1-5), Motivation (1-5), Capability (1-5). Response Probability = (A*M*C)/125.
   - Exchange Value (TN01): EV = Reference Value + Differential Value.
   - ECOMO (TN02): Development + Operating + Opportunity Costs vs Monetary + Psychological Value. NPV calculation.
   - Anti-Hallucination (TN11): Confidence levels HIGH/MEDIUM/LOW/UNCERTAIN. Every claim needs a verifiable source.
   - Shell Method (TN02): Generate 3 scenarios (offensive, defensive, transformative).
   - War Gaming (TN02): Blue Team, Red Team, Control, Debrief roles.
   DO NOT include any company-specific data. All data will be retrieved live by the agents.

2. Create config.py with a CIConfig dataclass containing:
   CASE CONFIGURATION (changeable via Settings):
     company_a_name (default: 'BBVA'), company_a_description (default: 'European traditional bank')
     company_b_name (default: 'Revolut'), company_b_description (default: 'UK-based neobank')
     sector (default: 'European Retail Banking')
     strategic_question (default: 'How serious is Revolut threat to BBVA in Spain?')
     market (default: 'Spain')
   LLM SETTINGS: model (claude-sonnet-4-20250514), temperature (0.3), max_tokens (4096)
   AGENT PROMPTS: system_prompt for EACH agent (collector, amc, scenarios, ecomo, verifier, reporter, wargame)
     Each prompt uses {company_a}, {company_b}, {sector}, {market} as template variables
     so they auto-adapt when the case is changed.
   THRESHOLDS: low_confidence_threshold (0.30), min_sources_for_high (2), max_react_cycles (5),
     discount_rate (0.10), projection_years (5)
   COLLECTIONS: list of ChromaDB collections

3. Create requirements.txt: anthropic, langchain, langgraph, chromadb, pydantic, fastapi, uvicorn, httpx

4. Create main.py FastAPI app with endpoints:
   GET /health — returns {status: ok, case: company_a vs company_b}
   GET /api/config — returns ALL config as JSON
   POST /api/config — updates config (partial updates OK)

```
   POST /api/analyze — placeholder {status: not_implemented_yet}

5. Create a Dockerfile that does EVERYTHING in one container:
   - FROM python:3.11-slim
   - Install Node.js (for building React frontend)
   - COPY requirements.txt and install Python deps
   - COPY frontend/ and run npm install + npm run build
   - COPY the rest of the Python backend
   - Copy frontend dist/ to a static/ directory
   - FastAPI serves static files at / and API at /api/
   - EXPOSE 8080
   - CMD uvicorn main:app --host 0.0.0.0 --port 8080
   This single container serves BOTH frontend and backend.

6. Create .gitignore: .env, __pycache__, .chroma/, node_modules/, dist/, .venv/

7. Create React frontend in /frontend with Vite + Tailwind CSS:
   Tab 1 - Overview: case description (reads from config), pipeline diagram, Run button
   Tab 2 - Settings with these GROUPS:
     CASE STUDY (most important — top of page):
       Company A name + description (default: BBVA)
       Company B name + description (default: Revolut)
       Sector + Market + Strategic Question
       A note: 'Change these to analyse ANY two competitors in ANY sector'
     GLOBAL LLM: model, temperature, max_tokens
     AGENT PROMPTS: system prompt text area for EACH agent (auto-filled with defaults using
{company_a} variables)
     SCORING: AMC rubrics (A/M/C text areas), min evidence per score
     SCENARIOS: num scenarios, default probabilities, reference scenario description
     ECOMO: discount rate, projection years
     ANTI-HALLUCINATION: LOW threshold, min sources for HIGH, max iterations
     PIPELINE: mode, active stages
     WAR GAME: Blue/Red/Control/Debrief prompts, num rounds, shock event
     REPORTER: system prompt, report template
   Tab 3-8: Placeholder tabs for future modules

8. When any case setting changes (company names, sector), all agent prompts auto-update
   their {company_a}, {company_b}, {sector}, {market} template variables.
```

Key Design: The Case Study is a Configuration, Not Code

The BBVA vs Revolut case is just the DEFAULT configuration. Students (or the professor) can change it to:

Company A: Zara Company B: Shein Sector: Fast Fashion Market: Europe
Company A: Tesla Company B: BYD Sector: Electric Vehicles Market: Global
Company A: Palantir Company B: Databricks Sector: Enterprise AI Market: US Government

When you change the case in Settings, ALL agent prompts automatically update.
The Collector searches for the NEW companies. The AMC Analyst scores the NEW competitors.
The War Game uses the NEW team profiles. The Reporter generates a brief about the NEW case.

This makes the system reusable for any CI scenario — not just one case study.

DEPLOY: Push to GitHub + Deploy to Google Cloud Run

Type into Claude Code:

Push all changes to GitHub with message 'Module 0: Foundation with Dockerfile, case selector, and Settings'.
Then build the Docker container and deploy to Google Cloud Run in region europe-west1
with --allow-unauthenticated. My Google Cloud project ID is [YOUR_PROJECT_ID].

Claude Code will build the React frontend, package it with the Python backend into one container, and deploy. You will get a URL like: https://bbva-revolut-ci-xxxxx-ew.a.run.app

Checkpoint 0: Foundation Deployed

[ ] Cloud Run URL shows the dashboard with all tabs
[ ] Settings tab shows Case Study section at the top with BBVA/Revolut as defaults
[ ] Changing company names in Settings updates the case description on Overview tab
[ ] /health returns {status: ok, case: 'BBVA vs Revolut'}
[ ] GitHub repository has all files including Dockerfile

2. Module 1: OSINT Collector with Live Data Retrieval and Validation

The Collector connects to **real data sources** via web search and public APIs. It does NOT use pre-loaded data. Every finding is retrieved live and validated for accuracy. If a source cannot be verified, it is flagged as LOW confidence.

CLAUDE CODE PROMPT 2

OSINT Collector with Live Search, Source Validation, and ChromaDB

Build the OSINT Collector agent that retrieves LIVE data and validates it.

AGENT: agents/collector.py
- Reads company_a, company_b, sector, market, strategic_question from config
- Uses ReAct framework: Think > Act > Observe > Loop (max cycles from config)
- RETRIEVAL: Uses Anthropic API with tool_use to search the web for REAL data about the configured companies
  Searches for: financial data, market share, executive statements, regulatory filings, product launches, hiring activity, news articles, analyst reports
- VALIDATION: For every finding, the agent must:
  1. Check if the source URL is a real, authoritative domain (company website, regulator, news outlet)
  2. Cross-reference key numbers against at least one other source when possible
  3. Flag contradictory data explicitly (store BOTH versions with UNCERTAIN confidence)
  4. Mark data freshness: reject data older than 18 months unless it is historical baseline
  5. Score confidence: HIGH (2+ authoritative sources agree), MEDIUM (1 reliable source), LOW (inference/weak source), UNCERTAIN (conflicting data found)
- Classifies each finding by AMC dimension: awareness, motivation, capability
- Stores validated findings in ChromaDB with metadata: source_url, date, confidence, company, amc_dimension
- Returns structured JSON with all findings and a data quality summary
- Uses the system prompt from config.py (which uses {company_a}, {company_b} template variables)

MEMORY: memory/vector_store.py
- Create 4 ChromaDB collections: {company_a}_intel, {company_b}_intel, market_context, historical (collection names adapt to the configured case)
- NO pre-seeded data. The Collector fills collections from live searches.
- Query interface returning top-5 relevant chunks with source metadata

TOOLS: tools/web_search.py + tools/source_validator.py
- web_search.py: wrapper around Anthropic API tool_use for web search
- source_validator.py: checks URL reachability, domain authority (is it .gov, .edu, company IR page, major news outlet?), date extraction, and basic content verification

ENDPOINTS:
  POST /api/agents/collector — runs Collector with live search, returns validated results
  GET /api/memory/search?q=...&collection=... — searches ChromaDB
  GET /api/memory/stats — document count per collection

FRONTEND: Update OSINT Data tab:
- Search bar to query ChromaDB
- Table of collected data with columns: Claim, Source URL (clickable link), Company, AMC Dimension,
  Confidence (colour-coded badge: green/yellow/orange/red), Date, Validated (checkmark/cross)
- Data Quality Summary: total findings, % validated, % HIGH confidence
- Collection statistics per collection
- 'Run Collection' button with loading spinner
- Each finding row is expandable to show the validation details (why this confidence level)

Configurable Settings: OSINT Collector

Collector System Prompt: (text area) Uses {company_a}, {company_b}, {sector}, {market} variables.
  Students can add specific data sources to search (e.g., 'Also check ECB banking statistics').
Max ReAct Cycles: (slider 1-10, default 5)
Search Breadth: (dropdown: narrow/balanced/wide) How many different queries to run.
Source Validation Strictness: (dropdown: lenient/standard/strict)
  lenient: accept any source, standard: require known domains, strict: require 2+ sources per claim
Minimum Source Authority: (dropdown: any/news/official/regulatory)
  Defines what counts as a 'reliable' source for MEDIUM confidence.
Data Freshness Cutoff: (months, default 18) Reject data older than this.

CI Connection: TN03 OSINT + TN11 Anti-Hallucination. The validation layer ensures the system never accepts unverified data. Students can tighten or loosen validation from Settings.

DEPLOY: Push to GitHub + Deploy to Google Cloud Run

Push to GitHub with message 'Module 1: OSINT Collector with live search and source validation'.
Rebuild and deploy to Google Cloud Run.
Test at Cloud Run URL: click 'Run Collection' and verify LIVE data appears with real source URLs.
Try changing the case to different companies in Settings and running Collection again.

Checkpoint 1: Live Data Collection Working

[ ] Collector retrieves REAL data from the web (not pre-loaded) with real URLs
[ ] Source validation badges show on each finding (green/yellow/orange/red)
[ ] Contradictory data is flagged as UNCERTAIN with both versions stored
[ ] ChromaDB collections are named after the configured companies
[ ] Changing companies in Settings and re-running produces data about the NEW companies

3. Module 2: AMC Analyst Agent

The AMC Analyst retrieves data from ChromaDB (collected live by Module 1) and produces scored assessments. It works with whatever companies are configured — not hardcoded to BBVA/Revolut.

CLAUDE CODE PROMPT 3

AMC Analyst with Evidence from Live Data and Radar Chart

Build the AMC Analyst agent.

AGENT: agents/amc_analyst.py
- Reads company_a, company_b from config
- Retrieves data from ChromaDB collections ({company_a}_intel + {company_b}_intel)
- Scores Company B vs Company A on Awareness (1-5), Motivation (1-5), Capability (1-5)
- For each score: numeric value, 2-3 evidence items with source URLs from ChromaDB, confidence level
- Validates that evidence actually supports the score (e.g., if score is 4/5, evidence must be strong)
- Calculates Response Probability = (A * M * C) / 125, adjusted for qualitative factors
- Predicts timeline for competitive response
- Uses scoring rubric and system prompt from config.py (template variables auto-replace)

ENDPOINT: POST /api/agents/amc

FRONTEND: Update AMC Analysis tab:
- Company names read from config (adapts to any case)
- Radar chart with A, M, C scores for both companies
- Evidence table with clickable source URLs
- Response Probability with explanation
- 'Run AMC Analysis' button

DEPLOY: Push to GitHub + Deploy to Google Cloud Run

Push with message 'Module 2: AMC Analyst with live evidence and radar chart'.
Rebuild and deploy to Cloud Run. Test: Run Collector then AMC. Verify scores based on real data.

Checkpoint 2: AMC Analyst Live

[ ] AMC scores based on REAL data retrieved by the Collector (not invented)
[ ] Evidence URLs are clickable and lead to real sources
[ ] Changing companies in Settings and re-running produces different AMC scores

4. Module 3: Scenario Planner, Exchange Value, and ECOMO

Three chained agents producing economically evaluated strategic options. Scenarios are generated based on the AMC analysis of whatever companies are configured.

CLAUDE CODE PROMPT 4

Scenarios + Exchange Value + ECOMO with Configurable Parameters

Build three chained agents:

AGENT: agents/scenario_planner.py
- Takes AMC output as input, reads company names and sector from config
- Generates configured number of scenarios (default 3)
  Scenarios are sector-appropriate: for banking they might be Digital Fortress / Acquire / Defend, but for retail fashion they would be completely different strategies
- Each has: name, description, probability, assumptions, risks

AGENT: agents/exchange_value.py
- Reference Value: what happens with no action (configured baseline in Settings)
- Differential Value per scenario based on AMC-predicted responses

AGENT: agents/ecomo_evaluator.py
- 5-year NPV at configured discount rate
- Five categories: Development, Operating, Opportunity Costs vs Monetary, Psychological Value
- Values based on REAL data retrieved by Collector (market size, revenue, costs from actual sources)
- Expected Value = Probability * NPV

ENDPOINTS: POST /api/agents/scenarios, /api/agents/exchange-value, /api/agents/ecomo

FRONTEND: Scenarios & ECOMO tab with scenario cards, probability sliders, ECOMO table, NPV bar chart, Go/No-Go badge. 'Run Strategic Analysis' chains all three.

DEPLOY: Push to GitHub + Deploy to Google Cloud Run

Push with message 'Module 3: Scenarios, Exchange Value, and ECOMO'.
Rebuild and deploy. Test full chain: Collector > AMC > Scenarios > EV > ECOMO.

Checkpoint 3: Strategic Pipeline Live

[ ] Scenarios are sector-appropriate (not generic)
[ ] ECOMO values reference real data from the Collector
[ ] Go/No-Go recommendation displayed
[ ] Changing discount rate or probabilities in Settings changes results

5. Module 4: Anti-Hallucination Layer

Verification agents that check everything against real sources. The Source Verifier re-checks URLs collected by Module 1. The test suite uses real claims from the current case (not hardcoded BBVA data).

CLAUDE CODE PROMPT 5

Confidence Scorer + Source Verifier + Bias Detector

Build three verification agents:

```
AGENT: agents/confidence_scorer.py
- Scores any claim: HIGH/MEDIUM/LOW/UNCERTAIN based on source count and authority
- Dynamically generates test cases from the CURRENT case data in ChromaDB
  (not hardcoded — adapts to whatever companies are being analysed)

AGENT: agents/verifier.py
- Re-fetches source URLs to verify they still exist and support the claim
- Checks for stale data (source changed since collection)
- Generates 20 test cases: 10 from real ChromaDB data + 10 fabricated versions
  (fabricated by modifying real claims: wrong numbers, fake events, impossible statements)

AGENT: agents/bias_detector.py
- Coverage balance between Company A and Company B (40-60%)
- Blind spot detection: which AMC dimensions have thin evidence

ENDPOINTS: POST /api/agents/confidence, /api/agents/verify, /api/agents/bias
           POST /api/eval/run — runs all + 5-check scorecard

FRONTEND: Evaluation tab with confidence pie chart, verification table, bias report,
5-check scorecard (Hallucination <5%, AMC Consistency >80%, ECOMO Accuracy <5%,
Bias 40-60%, Competitor Mention 0).
```

DEPLOY: Push to GitHub + Deploy to Google Cloud Run

Push with message 'Module 4: Anti-Hallucination with live source re-verification'.
Rebuild and deploy. Test: Run eval suite — target 10/10 real pass, 10/10 fake caught.

Checkpoint 4: Verification Live

[ ] Source Verifier re-checks REAL URLs from ChromaDB
[ ] Test cases are generated from current case (not hardcoded)
[ ] 5-check scorecard displays on Evaluation tab

6. Module 5: LangGraph Orchestration Pipeline

CLAUDE CODE PROMPT 6

LangGraph Pipeline with Quality Gates and Human-in-the-Loop

Build the LangGraph orchestration.

1. CIState dataclass: strategic_question, raw_data, amc_scores, scenarios, exchange_values, ecomo_results, confidence_map, low_confidence_ratio, report, recommendation, approved, human_feedback, iteration_count

2. Graph: collect > amc > scenarios > exchange_value > ecomo > verify > bias > human_review > report > learn

3. CONDITIONAL after verify: if low_confidence > threshold AND iterations < max: back to collect
4. HUMAN-IN-THE-LOOP after bias: pause, show summary, wait for Approve/Revise/Abort
5. LEARNING CYCLE after report: save to historical collection

ENDPOINTS: POST /api/pipeline/run, GET /api/pipeline/status, POST /api/pipeline/approve

FRONTEND Overview tab: pipeline diagram with current stage highlighted, progress bar, 'Run Full Analysis' button, human_review modal, pipeline log, previous runs list.

Configurable Settings: Pipeline

Pipeline Mode: full_auto / human_checkpoints / step_by_step
Quality Gate Threshold: (%, default 30%)
Max Iterations: (number, default 3)
Active Stages: (checkboxes) enable/disable individual stages

DEPLOY: Push to GitHub + Deploy to Google Cloud Run

Push with message 'Module 5: LangGraph pipeline'. Rebuild and deploy.

Checkpoint 5: Pipeline Live

[ ] All 10 stages complete in order
[ ] Quality gate loops back when confidence is low
[ ] Human-in-the-loop modal works
[ ] Pipeline uses LIVE data for the configured case

7. Module 6: War Game Simulator

CLAUDE CODE PROMPT 7

War Game with Configurable Teams and Shock Events

Build War Game in agents/war_game.py.

4 agents using company names and data from config:
BLUE TEAM ({company_a}): defend market position. Resources from Collector data.
RED TEAM ({company_b}): attack market. Resources from Collector data.
CONTROL: evaluate realism, inject shock from Settings.
DEBRIEF: summarise surprises, broken assumptions, recommended actions.

System prompts use template variables so they adapt to ANY case.
Agent resources are populated from REAL data in ChromaDB, not hardcoded.

ENDPOINT: POST /api/agents/wargame
FRONTEND: War Game tab with colour-coded chat transcript, shock highlight, debrief report.

DEPLOY: Push to GitHub + Deploy to Google Cloud Run

Push with message 'Module 6: War Game simulator'. Rebuild and deploy.

Checkpoint 6: War Game Live

[ ] Teams use the configured company names and real resource data
[ ] Shock event from Settings is injected at configured round
[ ] Changing to a different case produces a completely different war game

8. Module 7: CI Reporter and Executive Brief

CLAUDE CODE PROMPT 8

CI Reporter with Full Brief and Export

Build Reporter in agents/reporter.py.

Generates Markdown executive brief with: Executive Summary, AMC table, Scenarios + EV, ECOMO comparison, Go/No-Go recommendation, Confidence Map, Data Quality Warning (if needed),
Source Appendix with all URLs, War Game highlights (optional toggle in Settings).

RULES: Every claim cites a source. LOW/UNCERTAIN use qualifying language.
The report is about {company_a} vs {company_b} in {sector} — adapts to any case.

ENDPOINT: GET /api/report
FRONTEND: Executive Brief tab with rendered report, confidence badge, PDF + Word download.

DEPLOY: Push to GitHub + Deploy to Google Cloud Run

Push with message 'Module 7: Reporter with executive brief'. Rebuild and deploy.

FINAL DEPLOYMENT. Your complete CI system is live!

Checkpoint 7: System Complete!

[ ] Full pipeline runs end-to-end with LIVE data
[ ] Executive brief cites real sources
[ ] Change case to different companies > re-run > get a completely different report
[ ] All 8 dashboard tabs functional from Cloud Run URL
[ ] PDF/Word download works

CONGRATULATIONS: Your reusable CI agent system is complete!

9. Deliverables Checklist

#	Deliverable	Module	Verification
1	GitHub repo + CLAUDE.md + Dockerfile	0	github.com/YOU/bbva-revolut-ci
2	Settings panel with case selector + all parameters	0	Can change companies from browser
3	OSINT Collector with LIVE search + source validation	1	Returns real data with real URLs
4	AMC Analyst with radar chart + live evidence	2	Scores based on retrieved data
5	Scenario Planner (3 scenarios)	3	Sector-appropriate, probabilities sum to 1.0
6	Exchange Value Engine	3	EV per scenario using real market data
7	ECOMO Evaluator with NPV	3	Go/No-Go using real financials
8	Confidence Scorer + Source Verifier + Bias Detector	4	Re-verifies real URLs
9	5-check evaluation scorecard	4	>80% pass rate
10	LangGraph pipeline (10 stages)	5	Human-in-the-loop works
11	War Game Simulator	6	Teams use real company data
12	CI Reporter executive brief	7	PDF/Word export with real citations
13	Case adaptability test	All	Change case in Settings > re-run > different results
14	Everything on single Cloud Run URL	All	Open in any browser

Grading: What to Submit

1. GitHub URL: https://github.com/YOUR_USERNAME/bbva-revolut-ci
2. Cloud Run URL: https://bbva-revolut-ci-xxxxx-ew.a.run.app
3. Run TWO different cases through the system (e.g., BBVA/Revolut AND one of your choice)
4. 2-page individual reflection (30% of grade):
   What did you learn? Which module was hardest? How would you improve it?
   What are the limits of AI for CI? What happened when you changed the case?

10. Complete Settings Reference

Group	Setting	Type	Default	CI Framework
Case	Company A name	Text	BBVA	N/A
Case	Company A description	Text area	European traditional bank	N/A
Case	Company B name	Text	Revolut	N/A
Case	Company B description	Text area	UK-based neobank	N/A
Case	Sector	Text	European Retail Banking	N/A
Case	Market	Text	Spain	N/A
Case	Strategic question	Text area	How serious is Revolut...	TN01
LLM	Model	Dropdown	claude-sonnet-4-20250514	N/A
LLM	Temperature	Slider	0.3	N/A
LLM	Max tokens	Number	4096	N/A
Collector	System prompt	Text area	(uses {company_a}...)	TN03
Collector	Max ReAct cycles	Slider 1-10	5	TN10
Collector	Validation strictness	Dropdown	standard	TN11
Collector	Data freshness cutoff	Months	18	TN03
AMC	System prompt	Text area	(uses {company_a}...)	TN01 Sec 2
AMC	Rubric A/M/C	3 text areas	(see TN01)	TN01 Fig 2.1
AMC	Min evidence/score	Number	2	TN01
Scenarios	System prompt	Text area	(uses {sector}...)	TN02 Sec 3
Scenarios	Num scenarios	Dropdown	3	TN02
Scenarios	Default probabilities	3 sliders	40/25/35	TN02
EV	Reference scenario	Text area	(sector-specific)	TN01 Sec 4
ECOMO	Discount rate	% slider	10%	TN02 Sec 5

ECOMO	Projection years	Number	5	TN02 Sec 5
Anti-Halluc.	LOW threshold	% slider	30%	TN11 Sec 2.4
Anti-Halluc.	Min sources HIGH	Number	2	TN11
Anti-Halluc.	Max iterations	Number	3	TN11
Pipeline	Mode	Dropdown	human_checkpoints	TN11 Sec 5
Pipeline	Active stages	Checkboxes	All	TN11
War Game	Blue/Red/Control	Text areas	(uses {company_a}...)	TN02
War Game	Shock event	Text area	(sector-specific)	TN02 Sec 2.2
War Game	Num rounds	Slider 1-5	3	TN02
Reporter	System prompt	Text area	(uses {company_a}...)	TN01 Sec 5
Reporter	Report template	Text area	Markdown template	TN01

References

Chen, M.J. & Miller, D. (2012). Competitive Dynamics. Academy of Management Annals.
Zahra, S.A. & Chaples, S.S. (1993). Blind Spots in Competitive Analysis. AME.
Oriesek, D.F. & Schwarz, J.O. (2008). Business Wargaming. Gower.
Yao, S. et al. (2023). ReAct: Synergizing Reasoning and Acting. ICLR.
Lewis, P. et al. (2020). Retrieval-Augmented Generation. NeurIPS.
Wu, Q. et al. (2023). AutoGen. Microsoft Research.
Wang, X. et al. (2023). Self-Consistency. ICLR.
Shinn, N. et al. (2023). Reflexion. NeurIPS.
Ji, Z. et al. (2023). Survey of Hallucination. ACM Computing Surveys.
Ng, A. (2025). AI Agentic Workflows. DeepLearning.AI.
LangGraph. https://langchain-ai.github.io/langgraph/
ChromaDB. https://docs.trychroma.com/
EU AI Act (2024). Regulation (EU) 2024/1689.